GOSIM®

GLOBAL OPEN-SOURCE INNOVATION MEETUP

GOSIM Shenzhen 2026

10月16-17日 October 16-17　　深圳·南山伊敦酒店 Aden Hotel, Shenzhen

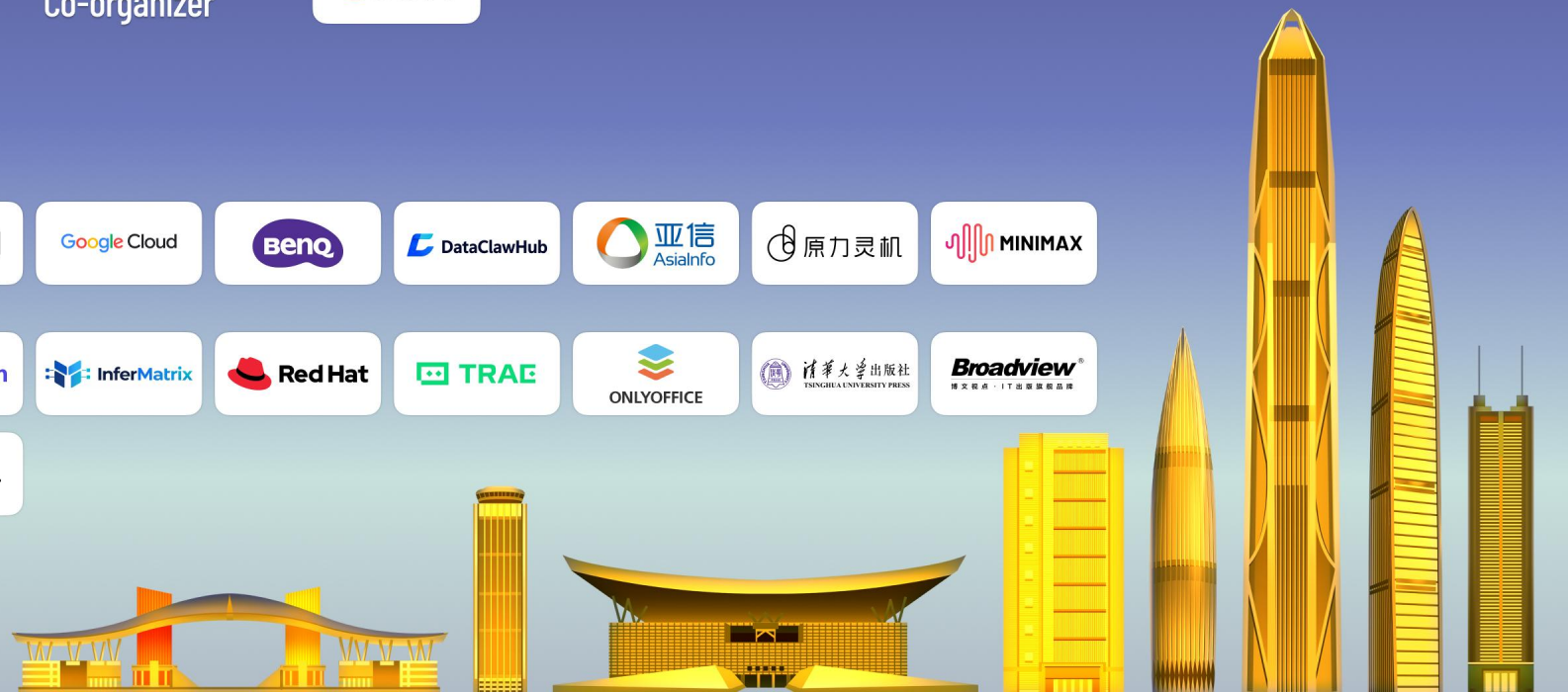
主办方 Host: GOSIM

承办方 Co-organizer: CSDN

白金赞助商 Platinum Sponsor: HUAWEI

金牌赞助商 Gold Sponsor: openJiuwen, KIMI, Google Cloud, BenQ, DataClawHub, 亚信 AsiaInfo, 原力灵机, MINIMAX

白银赞助商 Silver Sponsor: AMD, TaoToken, InferMatrix, Red Hat, TRAE, ONLYOFFICE, 清华大学出版社 TSINGHUA UNIVERSITY PRESS, Broadview 博文视点 · IT出版旗舰品牌, 清湛智造, CoHDL

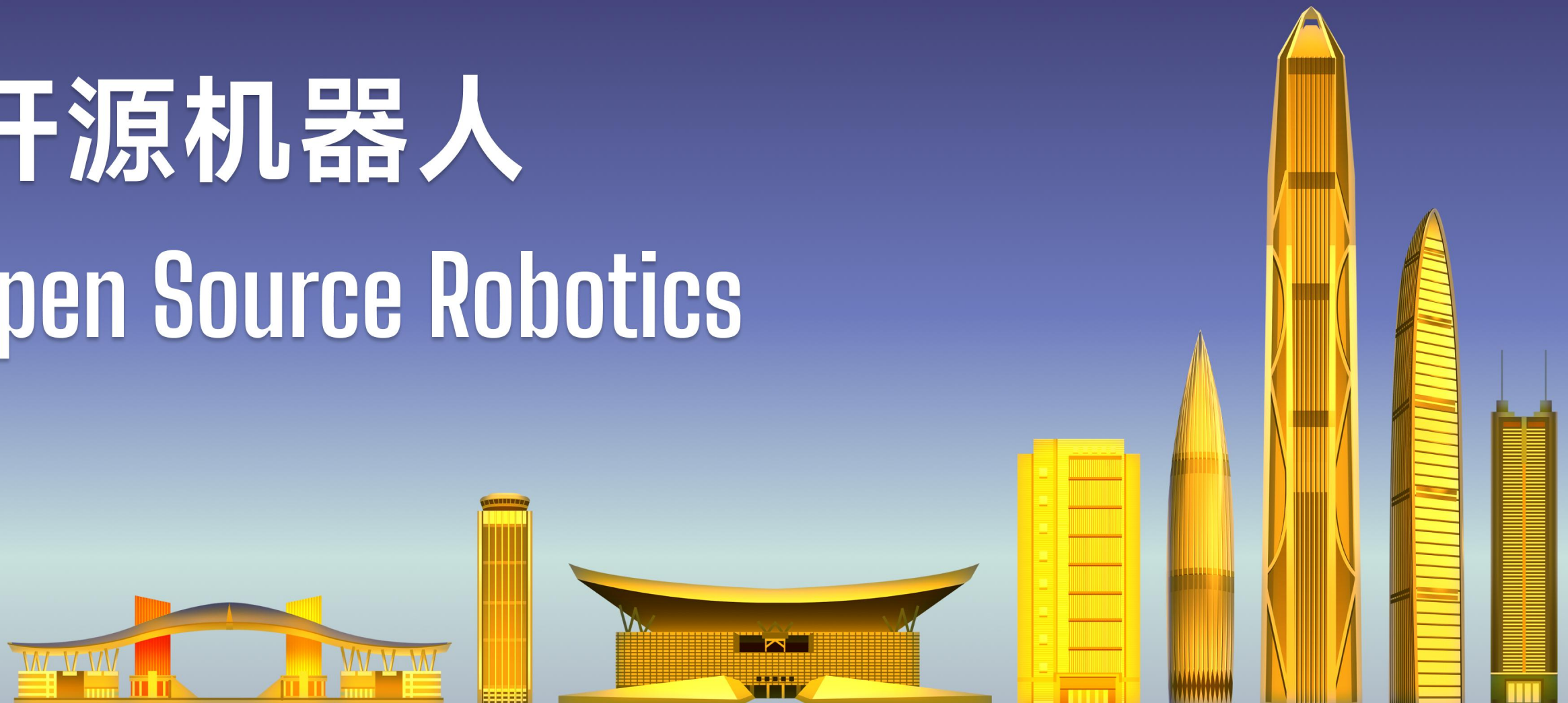


GOSIM®
GLOBAL OPEN-SOURCE INNOVATION MEETUP

GOSIM Shenzhen 2026

开源机器人

Open Source Robotics



GOSIM®
GLOBAL OPEN-SOURCE INNOVATION MEETUP

GOSIM Shenzhen 2026

Topic

使用DM0.5、OpenDM和Dexbotic构建开放式VLA技术栈

Engineering an Open VLA Stack with DM0.5, OpenDM, and Dexbotic



张海滨 Haibin Zhang

Dexmal AI基础设施与开源负责人

AI Infrastructure & Open Source Lead, Dexmal



GOSIM

GOSIM Shenzhen 2026

Engineering an Open VLA Stack

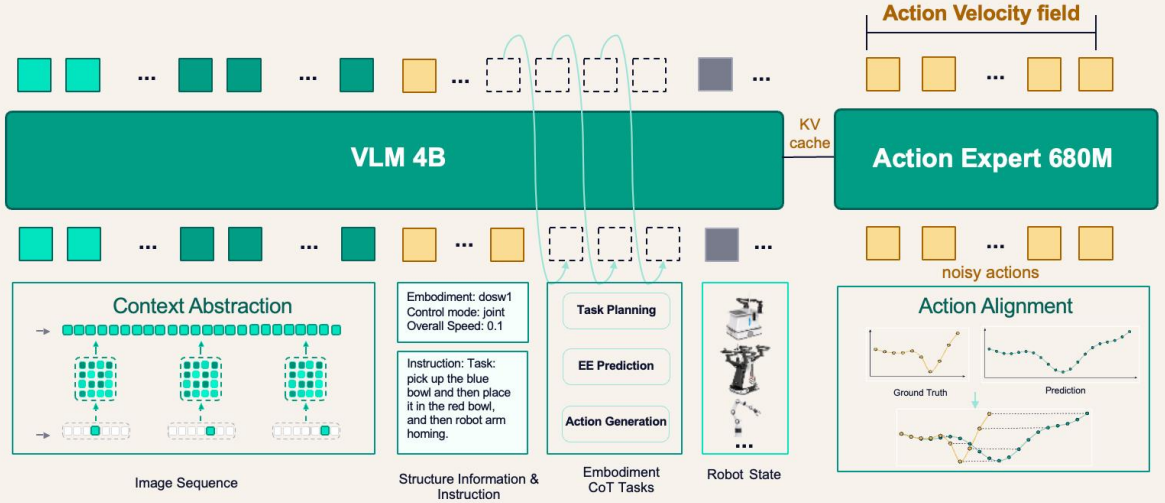
with DM0.5, OpenDM, and Dexbotic

张海滨

Dexmal | Infrastructure & Open Source Lead



DMO.5：长程记忆与具身推理



长程记忆

压缩约 60 秒视觉历史，结合当前观测判断任务进度

具身思维链

11 类任务联合动作监督，学习任务阶段与未来动作意图

动态轨迹对齐

单调匹配真实动作轨迹，减少示教节奏差异带来的噪声

公开评测登榜成绩

RoboColiseum 2026.09.14

四项第一

指令遵循、空间推理、鲁棒性、通用操作

RoboDojo-Sim 2026.08.25 · DM05-MEM

综合榜第一

综合得分 24.90 · 平均成功率 19.34%

RoboChallenge Table30 V2

真机评测 SOTA

成功率 43.0% · 综合得分 54.42

DMO.5 全生命周期工程架构

数据生产 → **模型训练** → **推理服务** → **模型评测**

执行链路

数据管理与供给	预训练与微调	API 与真机执行	仿真与真机评测
多源入湖 / DexIR	Checkpoint / 版本	动作 / 轨迹 / trace	任务回归 / 失败归因

评测失败按版本回流，补齐数据并触发下一轮优化

Agentic 工程实践

数据生产	训推优化	模型评测
找数据、发任务，预审预标 平台处理、质检与交付	策略检索与实验编排 验收结论回写策略库	高层规划、低层执行 独立评分与回流

工程平台

数据供给	训练框架	推理运行时	评测平台
Range / JSONL 索引 缓存 / CPU 混部	有效 Token / 融合算子 FSDP / HSDP	Rust / TensorRT CUDA Graph	任务版本 / 并行 Worker 轨迹与失败回流

基础设施

多云基础设施

开源与社区

内部验证后，选择性开放

工程验证与筛选

精度、性能与可复用性

OpenDM

DMO.5 权重 · 训推与评测代码

Dexbotic

具身智能开发框架

LeRobot

社区复现与贡献

多云资源支撑数据、训练、推理与评测，优秀工程实现反馈社区

面向百万小时级的 Agentic 数据生产

自然对话

直接说要什么数据、
要怎么处理

算法工程师

“找出办公室场景、双臂操作办公用品的数据”

“评测失败最多的任务，训练数据有多少小时”

数据工程师

“按 Ego v3 要求处理 202609 这批数据”

“昨晚被抢占的那批，按原 Batch 重跑”

Agent：把需求变成平台任务

- 按场景、技能、道具找数据
- 列出数据源、处理步骤和并发
- 人确认后才提交或取消
- 查进度、日志和运行报告

确认后提交 ↓

↑ 回看状态和报告

平台任务编排

父 Run 切分片，每片单独执行

AI 预审 原因与问题帧 → **AI 预标** 预标结果 → **质检** 质量记录 → **训练前处理** 按分片执行 → **版本发布** 清单与索引

转 DexIR → 去静止 → 规则清洗 → 校验 → 导出 JSONL → 建索引

DexIR 统一中间表达

时间轴、多视图、状态与动作

统一处理

720p、30 fps、H.264

弹性执行

混部 + 独立 CPU 资源
减少闲置，保证时效

状态与数据交付

每个分片跑到哪一步
日志和运行报告

可训练数据版本

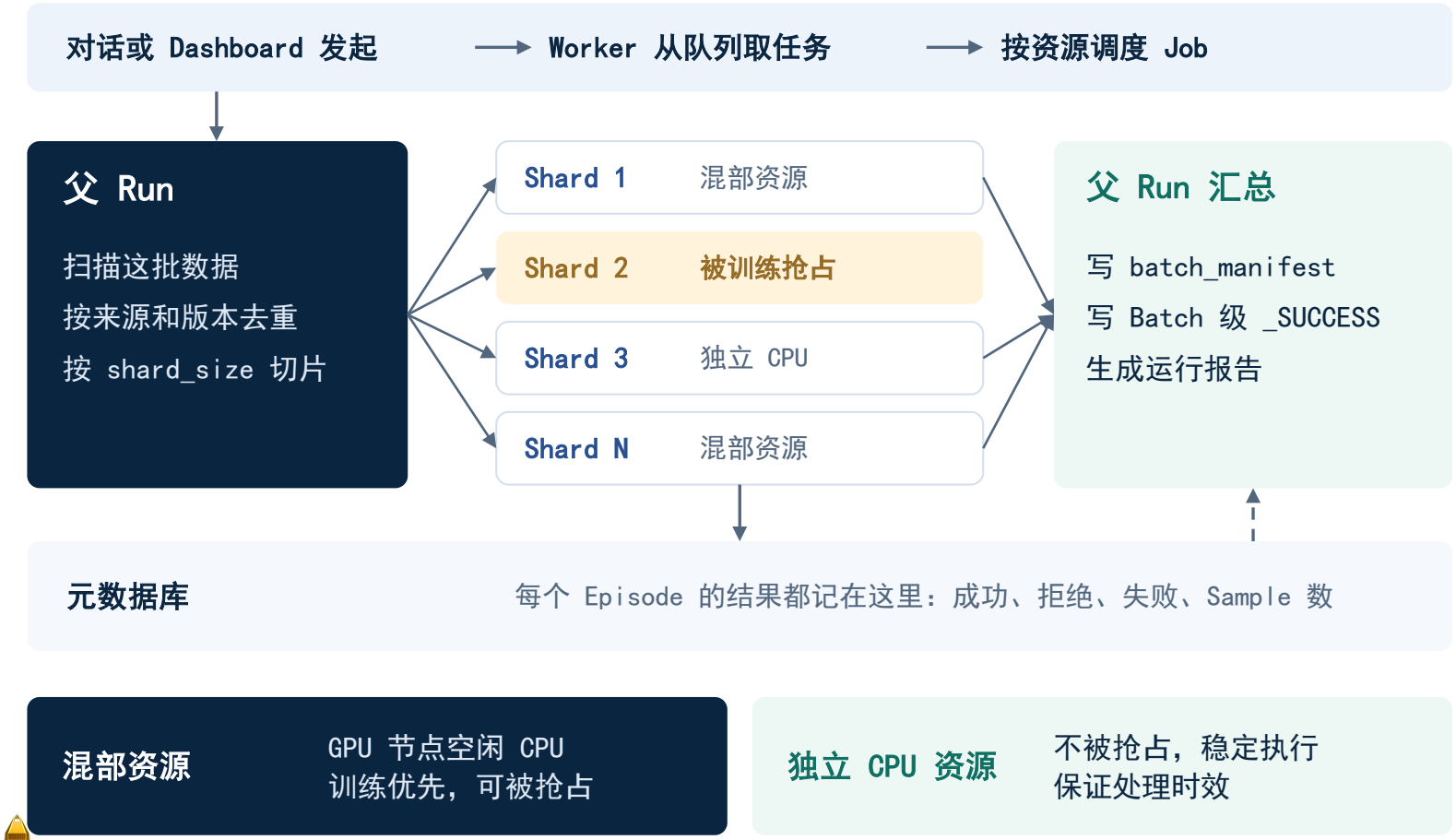
训练用 JSONL 和索引

Batch 成功 / 部分成功

拒绝与失败明细

调度显示完成，
不代表全部成功

混部与独立 CPU：任务原子化与断点重跑



任务原子化

细粒度分片

一个分片一个 Job
被抢占只影响这一片

状态持久化

每条数据的结果写入元数据库
不依赖进程内存，可随时查询进度

幂等重跑

按原 batch_id 重跑，沿用原分片
按来源与版本去重，同一数据只处理一次

故障隔离

单条异常记为拒绝
同一分片的其余数据正常处理

面向百万小时级的 Agentic 数据生产

自然对话

直接说要什么数据、
要怎么处理

算法工程师

“找出办公室场景、双臂操作办公用品的数据”

“评测失败最多的任务，训练数据有多少小时”

数据工程师

“按 Ego v3 要求处理 202609 这批数据”

“昨晚被抢占的那批，按原 Batch 重跑”

Agent：把需求变成平台任务

提交前先把参数给人看

- 按场景、技能、道具找数据
- 列出数据源、处理步骤和并发
- 人确认后才提交或取消
- 查进度、日志和运行报告

确认后提交 ↓

↑ 回看状态和报告

平台任务编排

父 Run 切分片，每片单独执行

AI 预审 原因与问题帧 → AI 预标 预标结果 → 质检 质量记录 → 训练前处理 按分片执行 → 版本发布 清单与索引

转 DexIR → 去静止 → 规则清洗 → 校验 → 导出 JSONL → 建索引

父 Run 去重、切片 → Shard 1 → Shard 2 被抢占后重跑 → Shard 3 1 条拒绝 → Shard N → 元数据库 逐条记结果

DexIR 统一中间表达

时间轴、多视图、状态与动作

统一处理

720p、30 fps、H.264

弹性执行

混部 + 独立 CPU

状态与数据交付

每个分片跑到哪一步
日志和运行报告

可训练数据版本

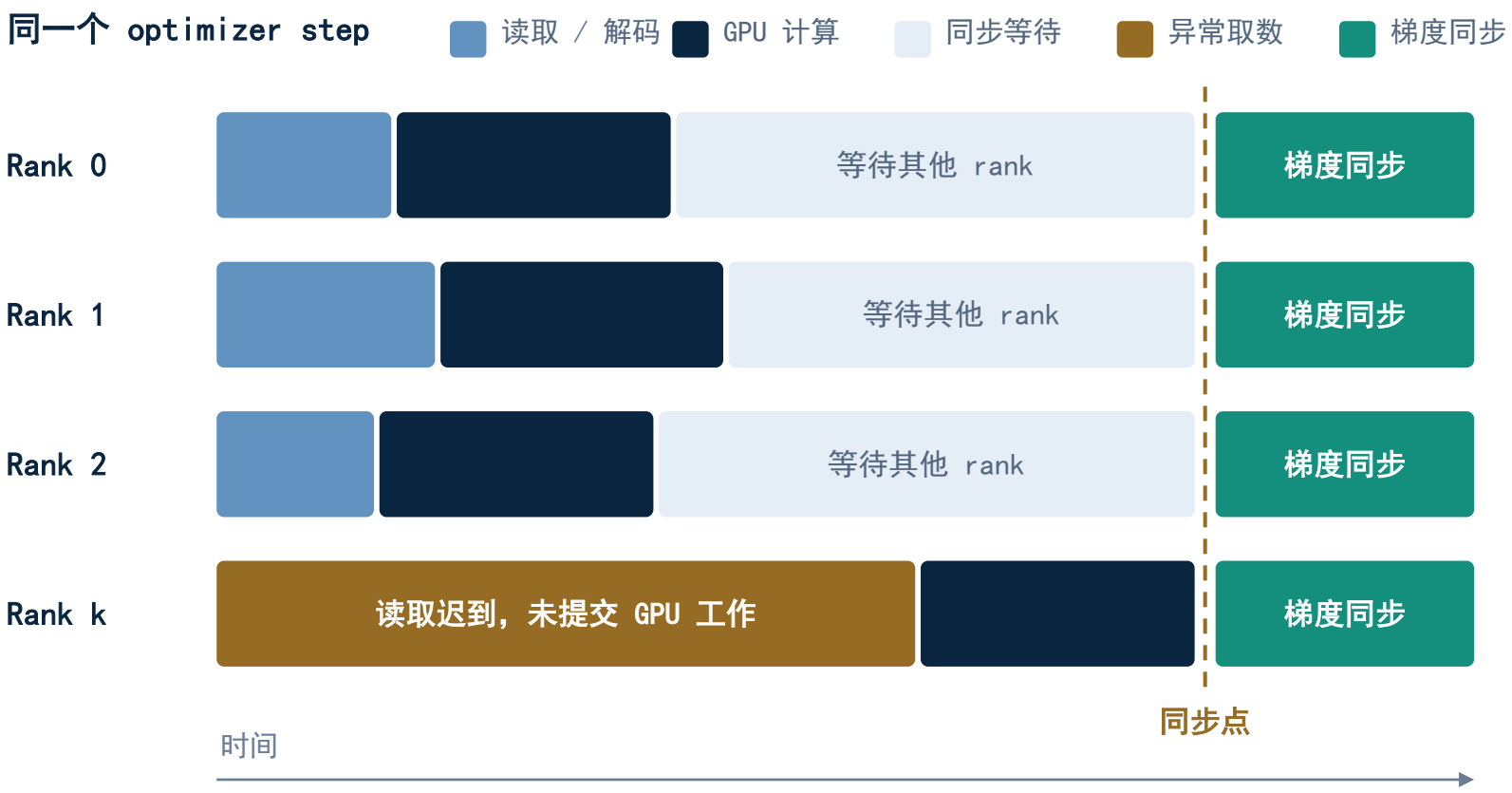
训练用 JSONL 和索引

Batch 成功 / 部分成功

拒绝与失败明细

**调度显示完成，
不代表全部成功**

输入长尾导致多卡训练等待



单 rank 最长 batch fetch

33.8 秒

batch fetch 超过 1 秒

6 次 / 80 步

一次 8 GPU、80 步诊断

NCCL 等待
也可能来自输入链路

训练数据供给：直读与按需缓存

训练数据清单
数据版本、分片与采样范围

→

数据预热
按容量与 lookahead 准备首窗、后续窗口

→

首窗就绪后启动训练
运行中持续准备下一窗口

OSS 对象存储

视频 / 图片 / 标注
数据版本

720p · 30 fps · H.264
图像按训练规格压缩存储

→

任务缓存后端（任选一种）

GPFS　共享工作集缓存

本机 NVMe　节点片段缓存

→

DataLoader

DVR / Range 取帧
JSONL offset 索引
异步读取与解码

样本准备队列

→

DMO.5

分布式训练
GPU 计算

未命中由数据服务回源

OSS 直读压缩片段

滑动窗口预取

首窗主动预热，运行时准备后续窗口；一个窗口包含多个 batch

主动预热	首窗 W0	预热 W1	预热 W2
训练消费		消费 W0	消费 W1

启动训练

缓存后端按任务选择，OSS 也可直接供给训练

视频、JSONL 与历史图像的读取优化

视频按需读取压缩片段



GOP 定位并解码，48/48 帧像素一致

平均响应字节
2.56 MB 降至 0.356 MB
−86.1%

OSS 读取优化，8 GPU
18.35 s 降至 5.7 s
3.22×

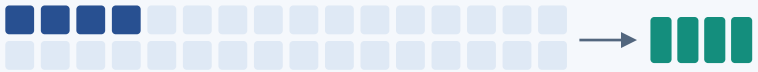
JSONL offset 索引与局部窗口

file / frame offset → mmap 索引 → seek + 64 行

10 亿行 offset，7.7 GiB 分片索引

整体吞吐（samples/s）
4.88 提升至 22.88
4.69×

历史图像：DataLoader 有效帧

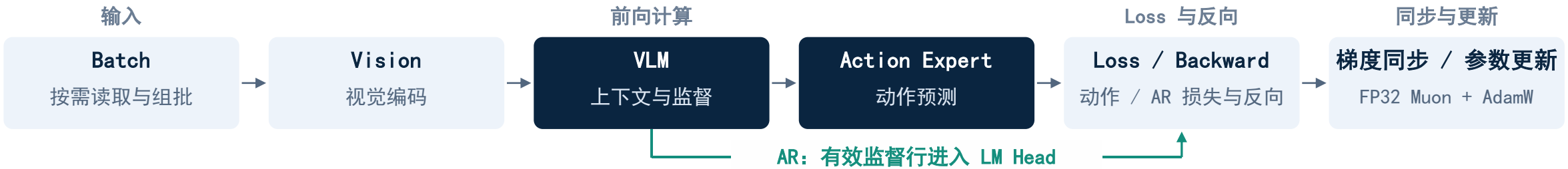


32 个槽位，仅 4 个有效；有效像素 + 原位置 / mask

队列张量缩小
588 降至 73.5 MiB / rank
8×

各 rank 取数耗时
18.6 ms 降至 4.24 ms
4.39×

DMO.5 的训练负载与优化措施



VLA 负载特征	训练中的具体问题	采用的工程措施
长历史与多相机输入	历史槽位稀疏，语言与历史长度不齐	**动态 Padding（16-token 对齐）** **Valid-token MLP gather / scatter**
动作 / AR 双监督	AR 监督行少，AE 反向重算开销大	**AR 有效监督行投影** **BF16 投影缓存与 AE 重算**
不同的 Attention 语义	Vision、VLM、动作三段的 mask 各不相同	**按语义选择 FA2 / Flex / cuDNN** **沿用 Liger GeGLU、RMSNorm、RoPE**
动态 Shape 与同步执行	新长度触发编译，慢 rank 拖住 collective	**Symbolic 长度复用** **结合逐 rank 时间线定位长尾**
跨阶段：供给与同步	取数长尾与节点间同步拖慢整步	**按需取数与异步重叠** **HSDP · 节点内 Muon 更新**

Valid-token MLP 与选择性重算

Valid-token MLP

MLP 逐行计算：先 gather 有效行，用同一个 GeGLU 计算，再 scatter 回原序列位置



Attention 不受影响

峰值吞吐 **+33.16%**　　峰值显存 / 卡 **−5.23 GiB**

8 GPU，320 步单变量对照

Selective BF16 Projection Cache

保留重算代价较高的 BF16 投影输出，反向时复用原投影输出

投影	保留字节 / 层	单层重算节省	
Down	118.75 MiB	9.107 ms	34 层
Query	95 MiB	2.069 ms	
Gate-Up	950 MiB	17.867 ms	4 层

B16 × L1520 单层预算；全模型短测选择 34 层 Down + 4 层 Gate-Up

每 step 节省 **381.1 ms**　　平均 step 提速 **+11.9%**

代价是每卡显存约 +7.90 GiB

动态长度编译

GPU 6 中超过 100 ms 的空闲段

10 段

最长一段

2.231 s

8 卡单步诊断：一个 rank 的暂停传给其他 rank



NCCL 时间长，是在等 GPU 6 编译

Symbolic 长度复用

在首次调用前声明动态长度，复用程序；运行时仍传入 BlockMask

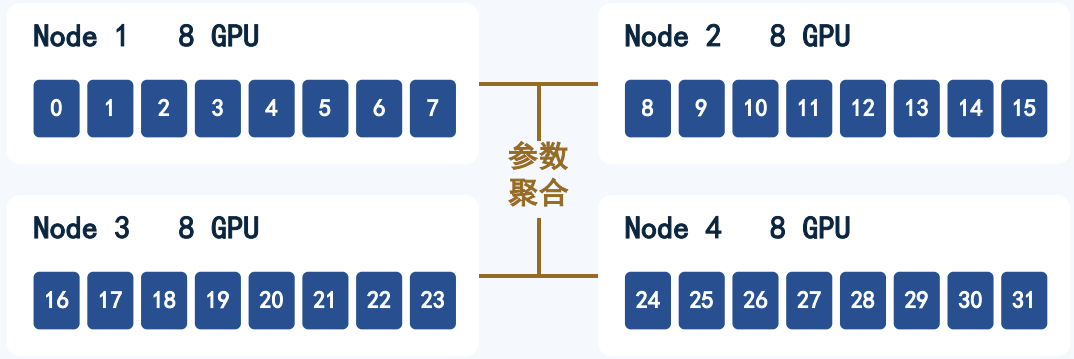


新长度首次调用：L 1520 改为 1488

原路径 **17.8 s** → 复用后 **10.6 ms**

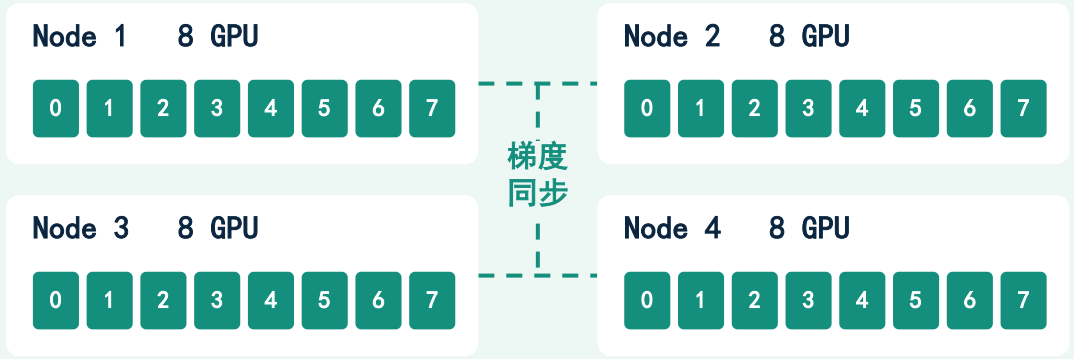
非 RDMA 集群上的 HSDP 优化

FSDP：32 卡全局分片



参数 all-gather 使用节点间网络

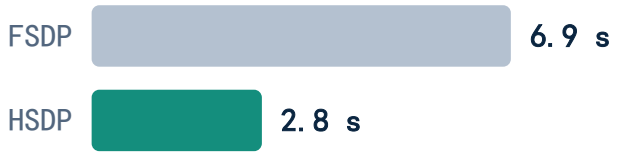
HSDP：节点内 8 分片，跨节点 4 副本



每节点 8 卡共同组成完整模型副本

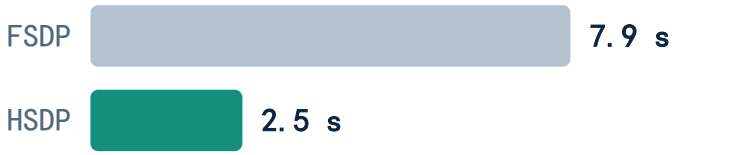
4 节点 × 8 卡 — 2.5×

E2E step 耗时



2 节点 × 8 卡 — 3.2×

E2E step 耗时



Muon 节点内组织

node_allgather

节点内 all-gather

node_owner

owner 路由与重复计算

DMO.5 推理优化架构

部署准备

模型与 Shape

采样、图数与长度配置

↓

目标卡编译产物

TRT Engine / AOT 算子包

↓

权重与工作区

打包权重，固定地址

↓

预热与 Capture

按桶捕获并校验数值

请求执行

请求预处理

图像、指令与状态

→ **视觉编码**

Vision + Projector

→ **VLM Prefix**

本次上下文 Prefill

→ **Action Expert × N**

动作采样、投影与 Mask

→ **API 输出**

Action Chunk

同一次请求：各采样步骤复用 Prefix KV

请求侧预处理

Rust Resize / CHW

归一化直接写 Tensor

视觉后端

FP16 TensorRT

固定图像数与分辨率

在 CUDA Graph 外执行

Prefix 专用路径

Packed QKV

QKNorm / RoPE 融合

FlexAttention

短序列采样内核

Time 条件预计算

Attention / GeGLU 专用路径

Norm / Residual 融合

动作后处理

反归一化

有效动作维度裁剪

打包 API 响应

OpenDM fast：静态执行与专用内核

Graph 外执行

Vision + Projector

静态 ONNX
FP16 TensorRT Engine

Graph replay

VLM Prefix
Packed QKV, FlexAttention

→

Suffix × 固定步数
Triton Attention / GeGLU

→

Action Chunk
固定输出地址

Prefix KV + 专用工作区复用

TimeMLP / AdaRMSNorm 时间条件预计算

投影与 elementwise 融合	Q/K/V 合成一次 GEMM · QKNorm、RoPE、Split 合并 · Norm 与 gated Residual 合并
按 Prefix 长度分桶	启动预热并 capture 多个 bucket · 输入拷入固定地址后 replay · 固定采样步数
有历史图像时仍用 fast 内核	历史视觉特征压成每帧 16 tokens · 超 bucket 安全走 eager 路径

600 ms 原始实现 PyTorch eager → **200+ ms** OpenDM default Suffix Graph 按长度分桶 → **99 ms** OpenDM fast TensorRT + 预捕获 Graph + Triton

SnapFlow：单步采样减少 AE 重复执行

Legacy Euler

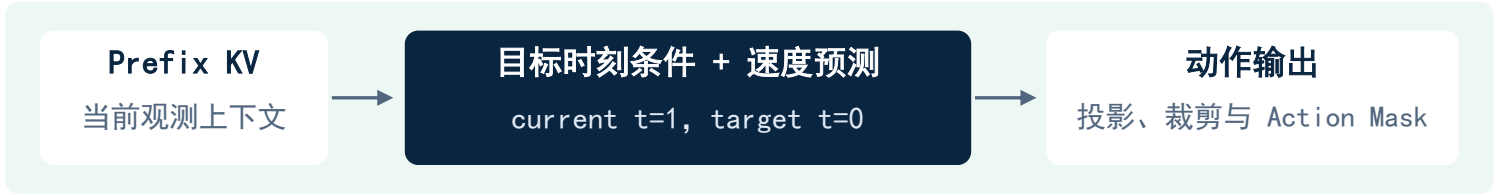
34 层 × 10 步



1 次 Prefix Prefill，再用 10 个 Suffix 步骤完成动作采样

SnapFlow

34 层 × 1 步



flow matching 训练与一致性目标结合，让模型学习 1 次速度预测就生成动作

API 推理延迟（完整配置）

1.69×

延迟降低 40.7%

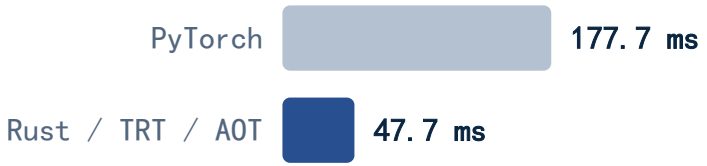


单步方法参考 Luan et al., SnapFlow, arXiv:2604.05656 (2026)

完整推理优化结果实测

同卡部署对照

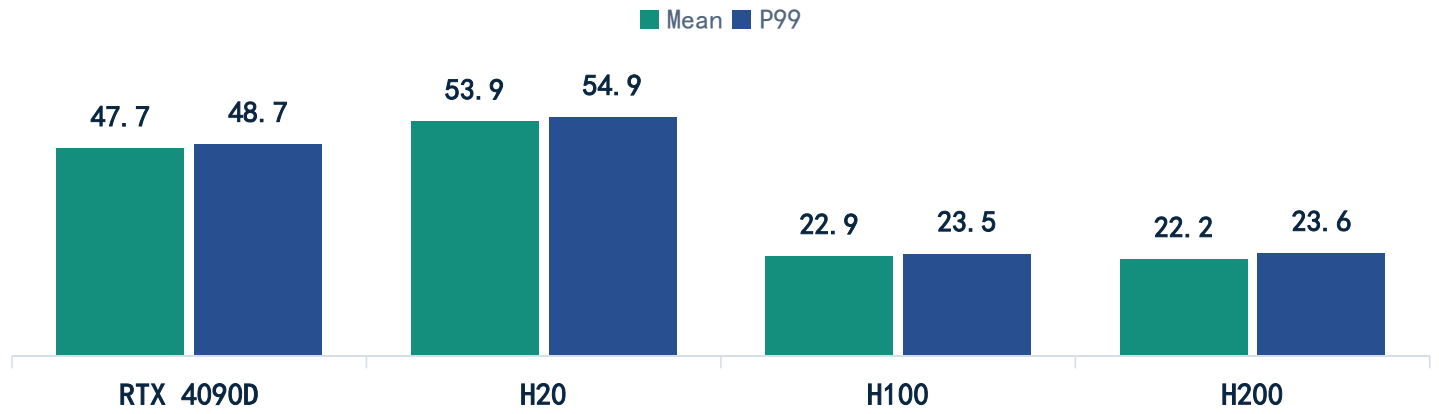
3.72×



完整部署组合：Rust 原生运行时 + TensorRT / AOT + SnapFlow 单步采样；4090D，Client API mean，ms

多 GPU 服务实测

Client API，预热 50 次，正式 1000 次，ms



LIBERO 汇总成功率，同一 Checkpoint

Rust runtime	98.5%	1970 / 2000
PyTorch	98.55%	1971 / 2000

Thor · 额外优化

52.7 ms

整模型 wall-clock P50

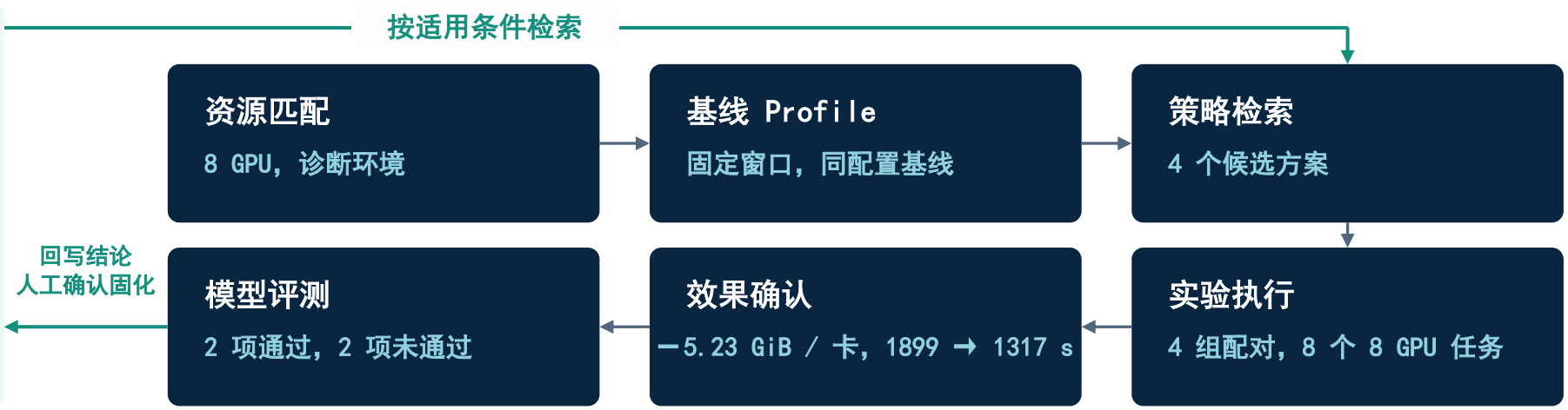
Prefix	Suffix	Vision / QK
FP8 投影 NVFP4 MLP	W8A8 短序列内核	MLP FP8 QK / RoPE 向量化

Agentic 训推优化：实验与策略

专家策略库

22 类

每类写明适用条件
和验收方法



09.07 实际记录：2 项通过诊断，2 项增量方案未通过

配对方案	实测结果	验收结论
Valid-token MLP	每卡显存 −5.23 GiB	通过单变量诊断
纯计算组合	训练进程 1899 s → 1317 s	通过组合诊断
历史图像传输 / 额外 KV0 缓存	供给改善 / 新增显存，整步增益不稳定	2 项增量方案未通过

335

云端任务登记

09.02 – 09.09
含基线、诊断、失败和排队记录

4 组

固定窗口配对验收

09.07
8 个完成的 8 GPU 任务

Agentic 模型评测：执行与评分

High-level 任务计划

目标与子任务
模型生成计划，自动下发

VLM 固定题集

图像、视频与问题
题集、问题与输入版本

评测编排

独立 Run：提交、查询与接续
固定模型、任务、环境与 seed，平台状态持久化

被测模型执行

仿真 Worker

LIBERO / 仿真环境

真机 Worker

Low-level 模型执行子任务

VLM 能力评测

对象与空间关系
动作时序
任务规划

持久化结果

视频、动作、状态
失败步骤与时间戳
产物 ID 与版本

VLM 答案与引用证据

答案、关键帧或时间段
输出的任务计划
题集与问题版本

独立评分

Code / VLM / Agent

逐项核验与证据引用
评分器版本独立保存

执行记录与评分记录分开管理

执行记录

模型在哪个环境、以哪个 seed 做了什么

评分记录

哪个规则和评分器作出了什么判断

结果复核：原子检查与证据核验

Agent Judge
组织工具与证据

检查项 目标与完成条件 → 结果证据 模型输出与执行记录 → 工具核验 查询、规则与回放

VLA 执行结果

示例任务：将红色方块放入托盘

原子检查项	判定	证据引用
拿起目标物体	True	状态记录：物体离台
物体进入托盘	True	位姿记录：进入目标区域
释放后持续稳定	Uncertain	视频末段遮挡，需复核
分步完成度	2 / 3	最终判定：待复核

基础评测器

Code Judge	位置、接触、数值阈值；规则确定，可重复
VLM Judge	看视频判断指令匹配和动作阶段，拿不准的交人工

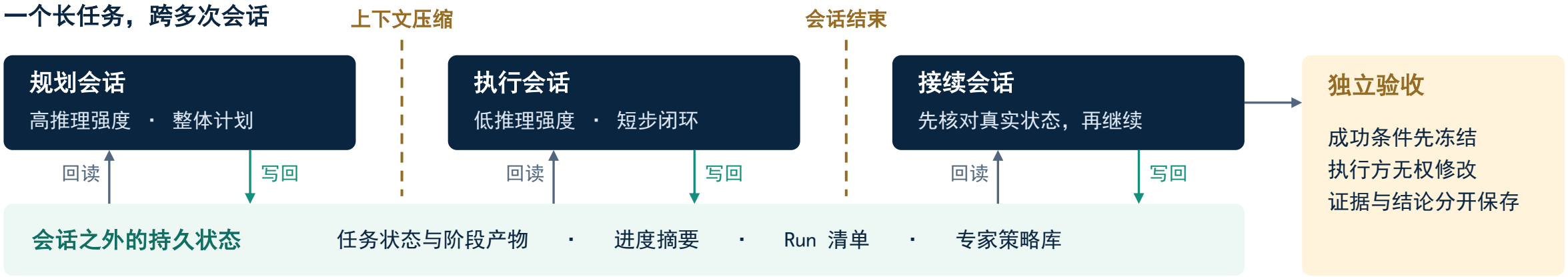
VLM 理解与规划输出

检查项与对照证据	
对象识别	模型答案、区域与参考标注
时序理解	时间段引用与阶段标注
任务规划	输出的计划与任务约束

校准与版本管理

固定校准集：覆盖成功、失败与遮挡
人工复核：保存分歧和仲裁记录
评分器版本：同一证据对比新旧评分

Agentic 工程的技术挑战



挑战	遇到的现象	我们的做法
模型选择与效率	逐步调用模型、全程高推理强度，又慢又贵	**关键规划用高推理强度，常规决策用低强度**
长任务遗忘	上下文压缩后丢失进度，重复提交任务	**每轮更新进度摘要，接续前核对真实任务状态**
跨会话的经验复用	会话结束，对话里的结论随之消失	**经验写入策略库按条件检索，复用前检查版本与输入**
工具权限与信息隔离	只靠 prompt 划边界，Agent 仍可能越权	**权限由程序限制，工具输出只当数据**

Agentic 工程：从一次执行到可复用的经验与资产，价值普惠



平台能力、长任务能力

基础能力
平台保存状态与阶段产物
会话结束后仍可查询和接续

接续前先查真实任务状态，再决定重跑还是继续

专家策略适用性

尤其冷启动阶段要有专家策略
按瓶颈、硬件和约束选策略
记录适用条件及未采用原因

NCCL 驻留高曾被读成通信瓶颈，实为等待编译

验证与验收

固定基线，独立验收
性能与模型质量达标后采用

局部编译方案在对照中变慢，数值校验也未通过，没有采用

可复用资产

迭代策略库
保留配置、脚本和验收记录
失败原因也进入可复用记录

09.07 两项未通过的方案，配置和原因一并留档

Thanks!

DMO.5 全生命周期 Agentic 工程实践

github.com/dexmal/OpenDM

github.com/dexmal/dexbotic

Q&A

GOSIM®
GLOBAL OPEN-SOURCE INNOVATION MEETUP

GOSIM Shenzhen 2026

开源机器人

Open Source Robotics

